



Project Gutenberg

Aisultan Sultangaziev

Diana Akhmetova

Azamatuly Khamza

Akan Beisen

PHASES OF PROJECT:

PHASE 1:

Web
Scraping
(Data
Collection)

PHASE 2:

Data Cleaning &
Transformation

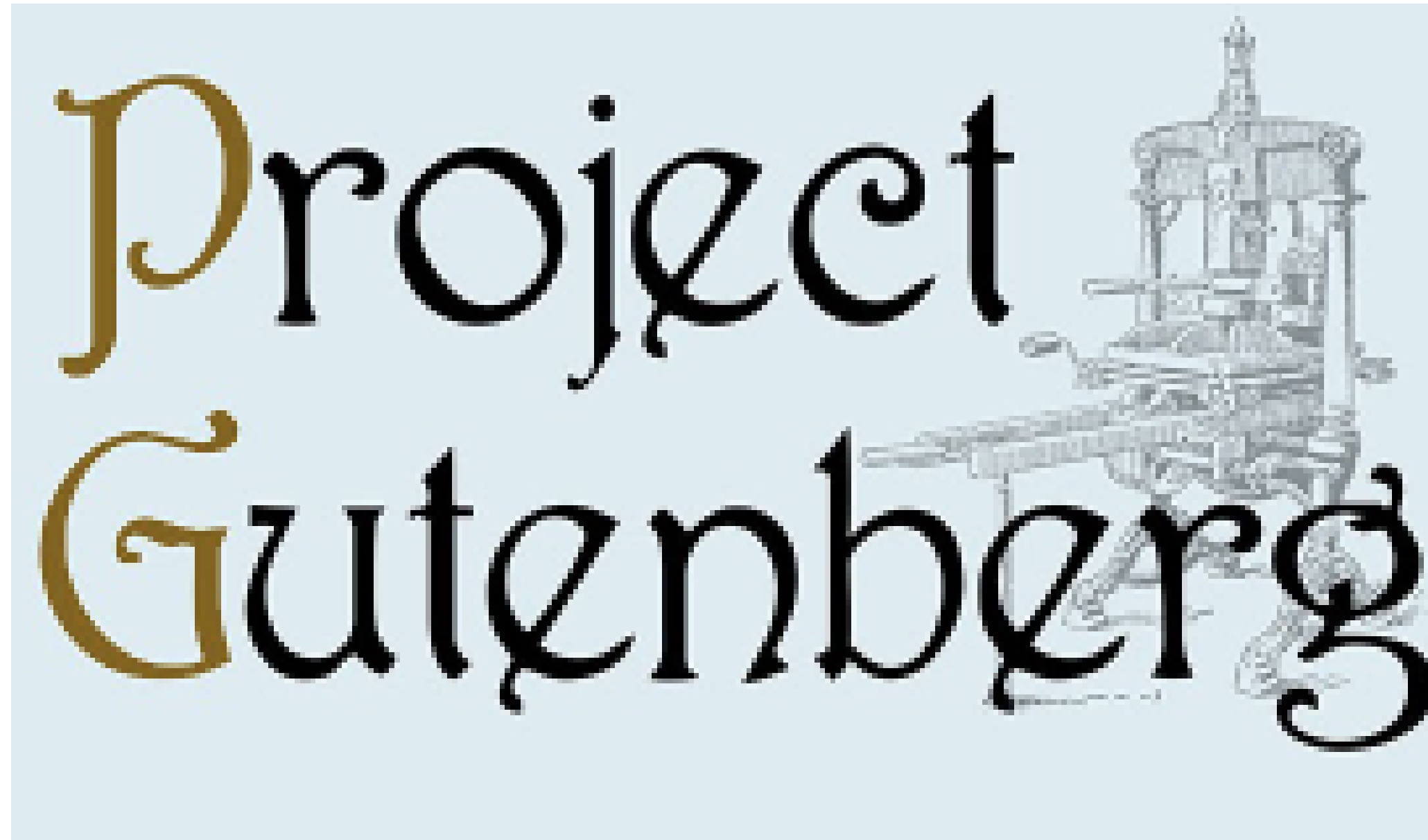
PHASE 3:

Exploratory
Data Analysis
(EDA)

PHASE 4:

Data Visualization

WHY THIS WEB SITE?



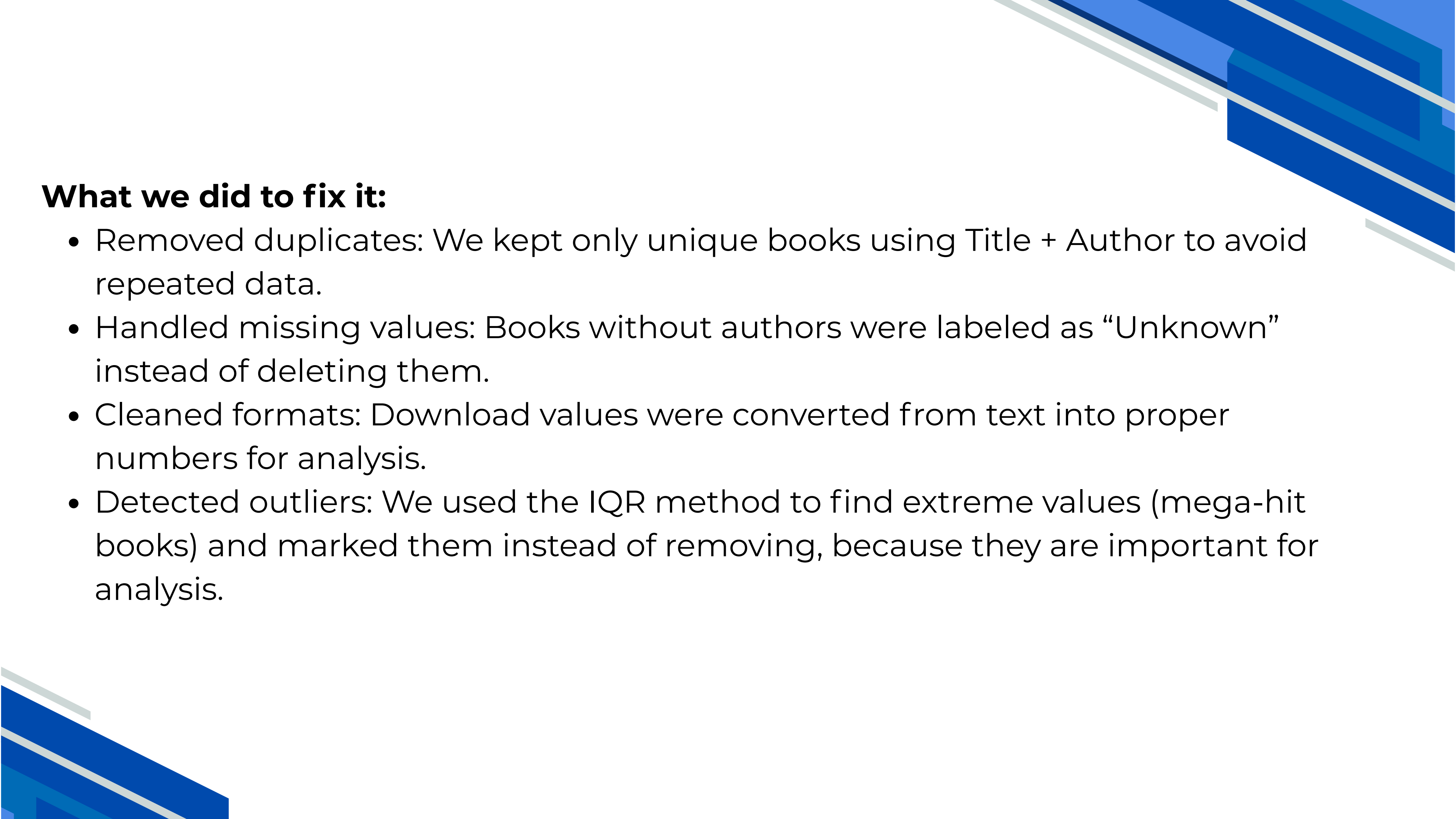
Why we chose this site ?

- It is one of the oldest and largest digital libraries with over 70,000 books, especially classics.
- The data is not directly available, so we had to build a web scraper to collect it from HTML pages.
- It provides download counts, which helps us analyze book popularity over time.
- The data is raw and inconsistent, which makes it good for practicing data cleaning and transformation skills.



```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
from scipy import stats
import plotly.express as px
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import logging
import random
from datetime import datetime
```

Phase 1: Web Scrapping (Data Collection)



What we did to fix it:

- Removed duplicates: We kept only unique books using Title + Author to avoid repeated data.
- Handled missing values: Books without authors were labeled as “Unknown” instead of deleting them.
- Cleaned formats: Download values were converted from text into proper numbers for analysis.
- Detected outliers: We used the IQR method to find extreme values (mega-hit books) and marked them instead of removing, because they are important for analysis.

THE SCRAPING ENGINE

Smart Automation We built a custom Python engine that systematically crawls Project Gutenberg. It doesn't just scan lists; it deep-dives into 5,000+ unique book pages to extract specific bibliographic metadata that isn't available in standard search results.

```
def fetch(self, url):
    try:
        time.sleep(random.uniform(0.8, 1.5)) # Respectful delay
        resp = requests.get(url, headers=self.headers, timeout=15)
        resp.raise_for_status()
        # ... parse response ...

    while len(self.data) < self.target_count:
        # Handle Pagination (Requirement)
        current_search_url = f"{self.search_url}&start_index={start_index}"
        soup = self.fetch(current_search_url)
        if not soup: break
```

```
try:
    # Metadata table parsing
    bibrec = soup.find('table', class_='bibrec')
    meta = {row.th.text.strip(): row.td.text.strip() for row in bibrec.find_all('tr') if row.th and row.td}
```

Ethical Execution To respect the platform, we integrated "Smart Delays" and custom headers. By mimicking human browsing patterns, the scraper avoids server strain, ensuring 100% data integrity without being blocked or throttled.

From Chaos to Structure The script isolates the "signal" from the "noise." It extracts messy HTML data—like download counts and release dates—and transforms it into a structured, analysis-ready dataset. Every book is captured with precision.

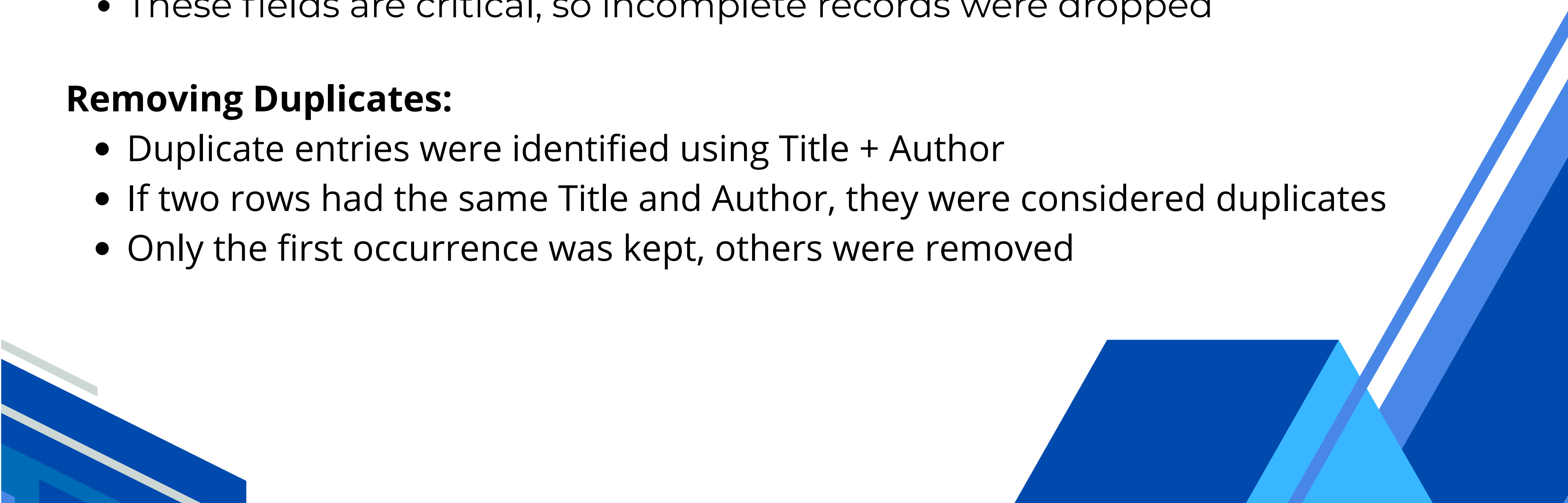
Data Cleaning & Preprocessing

Goal: To make the dataset clean, consistent, and ready for analysis.

Handling Missing Values:

- Missing values in the Author column were replaced with "Unknown"
- Rows with missing Title or Release Date were removed
- These fields are critical, so incomplete records were dropped

Removing Duplicates:

- Duplicate entries were identified using Title + Author
 - If two rows had the same Title and Author, they were considered duplicates
 - Only the first occurrence was kept, others were removed
- 
- The bottom right corner of the slide features several overlapping geometric shapes in various shades of blue, including triangles and parallelograms, creating a modern, abstract design.

Why this is important:

- Missing data can lead to incorrect analysis
- Duplicates can bias results and overcount records
- Cleaning ensures higher data quality and reliability

Result:

- Dataset became more accurate and consistent
- Ready for transformation and further analysis

```
... --- Comparison Table ---
      Metric  Before  After
0      Rows    5000   4853
1      Cols      9     16
2 Null Downloads  0      0
```

Feature Engineering

Definition:

Feature engineering is the process of creating new variables from existing columns to extract additional information and improve data analysis.

Implemented Features (from our code):

Year

- `df['Release_Date_Clean'].dt.year`
- Extracted the year from the release date
- Used for time-based trend analysis

Days_Online

- `(current date – Release_Date)`
- Measures how long a book has been available online
- Helps analyze relationship between age and popularity

Title_Length

- `len>Title.split()`
- Counts number of words in each book title
- Used in hypothesis testing (title vs downloads)



Tier (Popularity Segmentation)

- `pd.qcut(Downloads, q=4)`
- Divides books into 4 equal groups: Bronze, Silver, Gold, Platinum
- Helps compare performance between popularity levels

Author_Clean

- `Author.split(',')[0]`
- Extracts main author name only
- Improves grouping and visualization

Why we did this:

- Raw data was not enough for deeper insights
- New features allowed:
- Correlation analysis
- Hypothesis testing
- Better visualizations

Impact on Dataset:

- Increased columns: 9 → 16
 - Enabled EDA and statistical analysis
 - Made data more structured and analytical
- 

Exploratory Data Analysis (EDA)

Goal: To analyze the cleaned dataset and discover patterns, relationships, and insights.

Descriptive Statistics:

Calculated mean, median, mode, min, max, quartiles

Example:

Average downloads $\approx$ 6000

Max downloads $\approx$ 146,000

Shows overall distribution of data

Top 5 Books (by Downloads):

Frankenstein — Shelley

Moby Dick — Melville

Pride and Prejudice — Austen

Romeo and Juliet — Shakespeare

```
# 2. DESCRIPTIVE STATISTICS
```

```
# Mean, median, mode, and quartiles
```

```
numeric_summary = df[['Downloads', 'Days_Online', 'Title_Length']].describe()
```

```
numeric_summary.loc['mode'] = df[['Downloads', 'Days_Online', 'Title_Length']].mode().iloc[0]
```

```
print("--- 1. Stats ---\n", numeric_summary)
```

```
# 6. TOP RECORDS
```

```
# Identify 5 most popular books
```

```
top_books = df.nlargest(5, 'Downloads')[['Title', 'Author_Clean', 'Downloads']]
```

```
print("\n--- 5. Top 5 Books ---\n", top_books)
```

--- 1. Stats ---

	Downloads	Days_Online	Title_Length
count	4853.000000	4853.000000	4853.000000
mean	6000.973831	5516.113744	9.757264
std	6452.678152	2456.191414	3.032255
min	2756.000000	12.000000	1.000000
25%	3357.000000	4066.000000	7.000000
50%	4197.000000	6320.000000	10.000000
75%	6406.000000	6867.000000	12.000000
max	146193.000000	19868.000000	18.000000
mode	2988.000000	6202.000000	9.000000

--- 5. Top 5 Books ---

	Title	Author_Clean	Downloads
0	Frankenstein; or, the modern prometheus by Mar...	Shelley	146193
1	The City of God, Volume I by Saint of Hippo Au...	Augustine	139919
2	Moby Dick; Or, The Whale by Herman Melville	Melville	118655
3	Pride and Prejudice by Jane Austen	Austen	103210
4	Romeo and Juliet by William Shakespeare	Shakespeare	77436

```

# 7. HYPOTHESES
print("\n--- 6. Hypothesis Testing ---")

# H1: English books popularity
eng = df[df['Language'] == 'English']['Downloads']
others = df[df['Language'] != 'English']['Downloads']
t_stat, p1 = stats.ttest_ind(eng, others, equal_var=False)
print(f"H1 (English vs Others): {'Confirmed' if p1 < 0.05 else 'Rejected'} (p={p1:.4f})")

# H2: Title length vs Downloads
corr_t, p2 = stats.pearsonr(df['Title_Length'], df['Downloads'])
print(f"H2 (Title Length): {'Confirmed' if p2 < 0.05 else 'Rejected'} (Corr={corr_t:.4f})")

# H3: Longevity (Age) vs Downloads
corr_a, p3 = stats.pearsonr(df['Days_Online'], df['Downloads'])
print(f"H3 (Longevity): {'Confirmed' if p3 < 0.05 else 'Rejected'} (Corr={corr_a:.4f})")

```

--- 6. Hypothesis Testing ---

H1 (English vs Others): Confirmed (p=0.0001)

H2 (Title Length): Confirmed (Corr=-0.0468)

H3 (Longevity): Confirmed (Corr=0.1474)

Hypothesis Testing:

H1: English books more popular →

H2: Title length affects downloads →

H3: Older books have more downloads →

Data Visualization

Goal:

To visually explore the dataset and clearly present patterns, trends, and relationships.

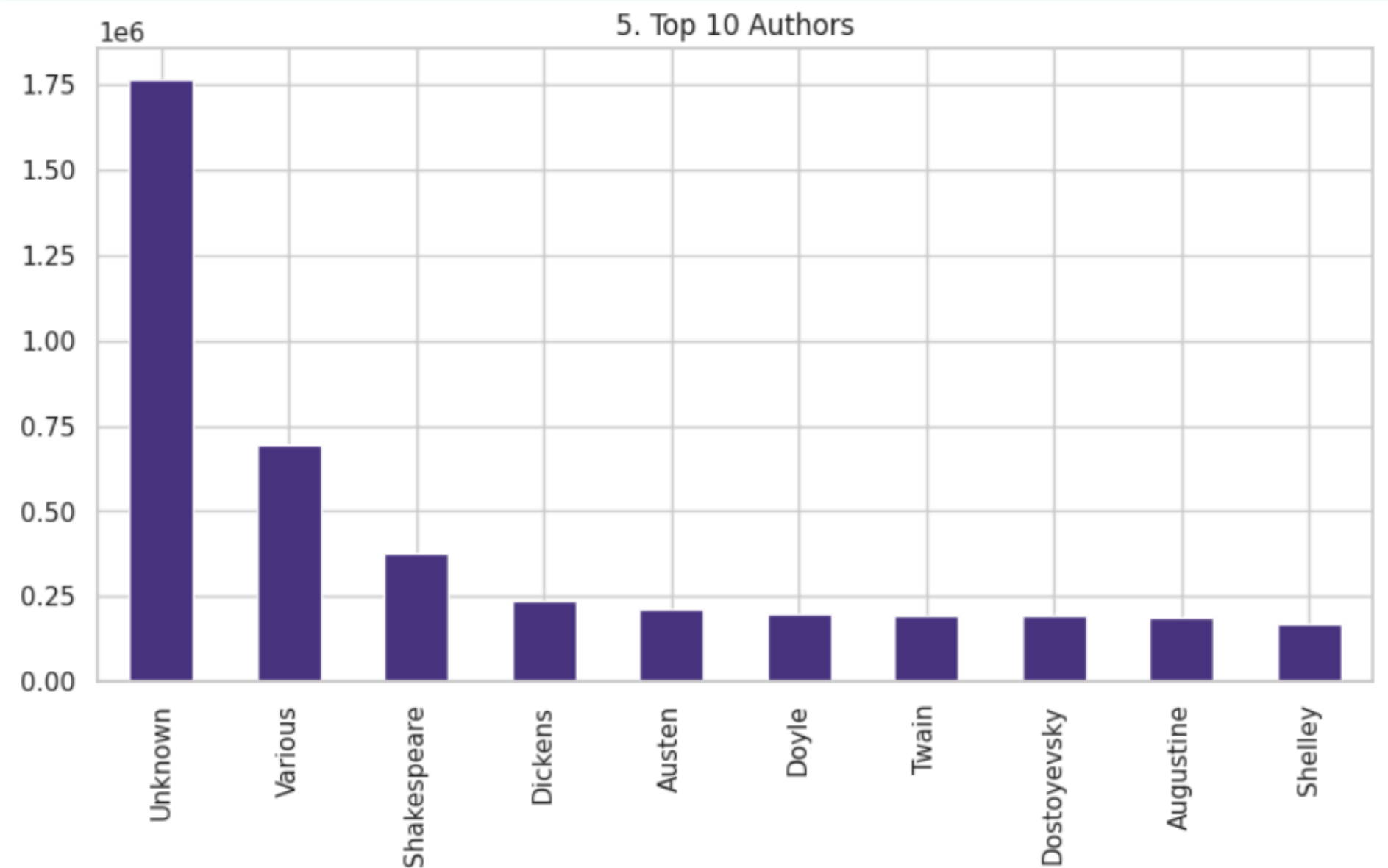
Tools Used:

- matplotlib
- seaborn
- plotly

Isn't it getting boring?
Let's get to the most
interesting part

Visualization 5 — Top 10 Authors

Library used: Matplotlib



What it shows:

- Total downloads grouped by each author
- Top 10 most popular authors in the dataset

• Result / Insight:

- A few authors contribute most of the total downloads
- Popularity is concentrated in classic literature authors
- Shows “who drives traffic” in the dataset

Visualization 10 — Regression Plot

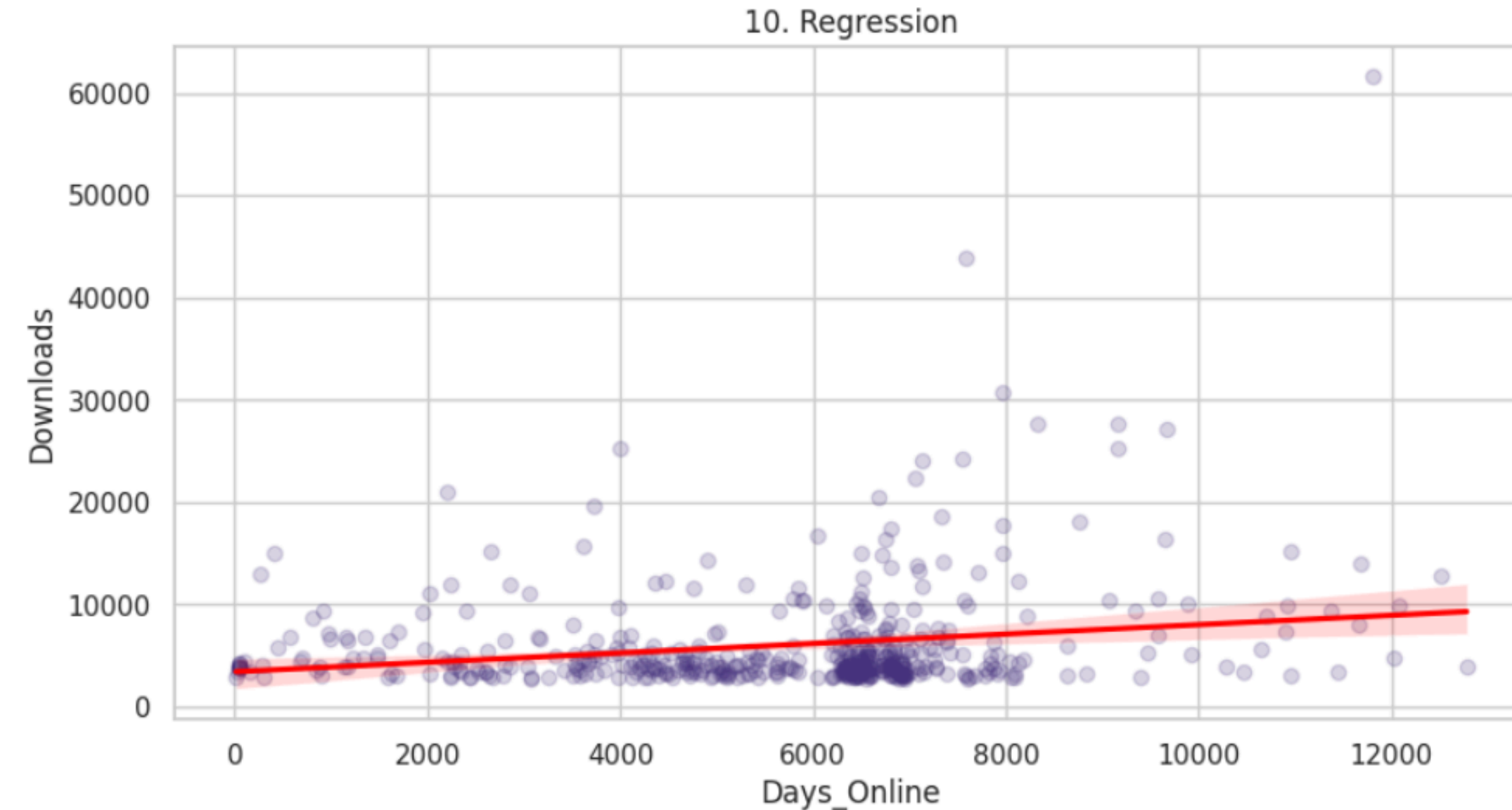
Library used: Seaborn

What it shows:

- Relationship between:
 - Days Online (book age)
 - Downloads (popularity)
- Includes regression line to show trend

Result / Insight:

- Slight positive relationship exists
- Older books tend to have slightly more downloads
- But correlation is weak, not very strong



Visualization 12 — Top 10 Books

Library used: Plotly

12. Top 10 Books



Result / Insight:

- Classic books dominate top positions
- Example: Frankenstein, Moby Dick, Pride and Prejudice
- Users can interact and explore data dynamically

What it shows:

- Top 10 most downloaded books
- Interactive bar chart (hover to see details like author)

**Thank
You!!**

